

NATIONAL TEACHERS COLLEGE**STUDENT DOCUMENTATION WORKSHEET****NTC_CC105 – Data Structure w/Lab | Terminal Assessment, Weeks 1–5****“Smart Campus Student Resource Management System”**

This worksheet accompanies the NTC_CC105 Terminal Assessment (Weeks 1–5). Complete every section as a group, then submit this worksheet together with your source code, GitHub repository, and screenshots. Refer to the Terminal Assessment document for full requirement details.

Group No.: GROUP 7**Section: BSIT 2.4****Date:****Instructor:****Group Members and Assigned Positions**

No.	Full Name	Assigned Position
1	Villalino, Dustin S.	Group Leader
2	Protacio, Santino Adelrey R.	Assistant Leader
3	Delos Reyes, Cedric James D.	Member
4	Buacan, Keny D.	Member
5		

SECTION 1 – ADT IDENTIFICATION (Requirement 1)

Identify at least two Abstract Data Types represented by your system.

ADT 1

ADT Name	List ADT
Data Represented	Represents an ordered, index-based collection of linear elements where each item maintains a specific sequential position relative to other elements in the list.

Operations	• <code>insert(index, element)</code> / <code>push_back(element)</code>: Adds an item at a specified index or appends it to the end. • <code>remove(index)</code> / <code>erase(position)</code>: Deletes an element from a specific index. • <code>get(index)</code> / <code>at(index)</code>: Retrieves the element stored at the given positional index. • <code>size()</code>: Returns the total number of elements currently stored in the list. • <code>isEmpty()</code>: Checks whether the collection contains any data items.
Purpose	To store, manage, and sequentially process a dynamically growing collection of data entries where maintaining order and positional retrieval are essential for system functionality.
C++ Implementation Used	<code>std::vector<T></code> (or Dynamic Array / Singly Linked List)

How is this ADT different from the actual data structure used to implement it?

The List ADT defines the high-level logical interface and operational rules (what operations can be performed, such as adding, deleting, or accessing elements by index) without specifying internal storage details. Conversely, the concrete Data Structure (such as `std::vector`) defines the low-level physical implementation in memory. It handles contiguous memory allocation, dynamic capacity resizing, element shifting during insertions/deletions, and runtime time complexity tradeoffs (e.g., $O(1)$ random access vs. $O(n)$ insertion).

NTC_CC105 – Student Documentation Worksheet

ADT 2

ADT Name	Map / Dictionary ADT
Data Represented	Represents an associative collection of unique key-value pairs, where each key serves as a distinct identifier linked directly to a specific value or record.
Operations	Represents an associative collection of unique key-value pairs, where each key serves as a distinct identifier linked directly to a specific value or record.
Purpose	To provide high-performance, fast lookup, updating, and indexing of records using unique identifiers (such as User IDs or codes) instead of performing slow linear searches.
C++ Implementation Used	<code>std::unordered_map<Key, Value></code> (Hash Table) or <code>std::map<Key, Value></code>

How is this ADT different from the actual data structure used to implement it?

The Map ADT establishes the conceptual model of key-value mapping and lookup operations, abstracting away internal data organization. The actual Data Structure used to implement it—such as `std::unordered_map` (a Hash Table)—provides the concrete execution logic. It uses a hash function to map keys to bucket array indices and manages hash collisions, achieving average $O(1)$ time complexity. Alternatively, using `std::map` implements a balanced binary search tree (Red-Black Tree) to maintain sorted keys with $O(\log n)$ search time.

SECTION 2 – STATIC ARRAY (Requirement 2)

Describe how your group implemented the required static array component.

Array Name / Purpose	Name: itemList (or systemLogs) Purpose: Serves as a fixed-size contiguous memory container to store and manage system data records whose maximum capacity is predefined at compile-time. It acts as the underlying concrete data structure for the system's List ADT.
Declaration & Initialization	Declared with a fixed constant size using C++ standard array syntax alongside an integer variable to track active entries: <pre>const int MAX_CAPACITY = 100; Item itemList[MAX_CAPACITY]; int currentCount = 0; // Tracks the number of inserted</pre>
Traversal Method Used	Implemented using a standard indexed for loop that iterates sequentially from index 0 up to currentCount - 1 to access or display each record: <pre>for (int i = 0; i < currentCount; i++) { cout << itemList[i].getDetails(); }</pre>
Searching Method Used	Implemented using Linear Search. The system iterates sequentially from index 0 to currentCount - 1 and compares the search key against each stored element: <pre>int search(int targetId) { for (int i = 0; i < currentCount; i++) { if (itemList[i].id == targetId) return i; // Found at index i } return -1; // Not found }</pre>
Updating Method Used	Direct array subscript assignment. After locating the target element's index via Linear Search, its fields are updated directly in O(1) time complexity: <pre>int index = search(targetId); if (index != -1) { itemList[index].name = newName; }</pre>
Bounds-Checking Method Used	Uses explicit conditional checks before insertion and array access to prevent buffer overflow and out-of-bounds runtime errors: • Insertion Check: if (currentCount < MAX_CAPACITY) before adding a new item. • Access Check: if (index >= 0 && index < currentCount) before reading or updating an element at a specified index.

SECTION 3 – DYNAMIC MEMORY (Requirement 3)

Describe how your group implemented pointers and dynamic memory allocation.

Pointer Declaration(s) Used	Declared dynamic pointers to reference memory allocated on the heap: <code>Item* dynamicList = nullptr;</code> (for dynamic array) or <code>Node* head = nullptr;</code> (for linked list implementation)
Purpose of Dynamic Allocation	To allocate memory at runtime on the heap rather than compile-time on the stack. This allows the application to handle variable data sizes dynamically, expand storage capacity on demand, and avoid stack overflow issues.
Where new Is Used	Used during system initialization or when resizing the dynamic collection: • Dynamic Array: <code>dynamicList = new Item[capacity];</code> when creating or resizing the array. • Linked List: <code>Node* newNode = new Node(data);</code> when inserting a new element.
Where delete / delete[] Is Used	Used when removing individual elements, resetting data structures, or inside class destructors to release allocated heap memory: • Single Object: <code>delete ptr; ptr = nullptr;</code> • Dynamic Array: <code>delete[] dynamicList; dynamicList =</code>
How Memory Leaks Are Prevented	1. Dangling Pointer Prevention: Immediately setting pointers to <code>nullptr</code> right after freeing memory with <code>delete</code> or <code>delete[]</code>. 2. Destructor Cleanup: Implementing proper destructors (<code>~ClassName()</code>) to ensure all dynamically allocated memory is deallocated when objects go out of scope. 3. Reallocation Safety: Deallocating old array memory before assigning a new memory block during array resizing.

SECTION 4 – SEARCHING (Requirement 4)

Search Algorithm Used	Linear Search (Sequential Search) (or Binary Search if data is sorted)
Best-Case Situation	Occurs when the target element being searched for is located at the very first index (index 0) of the data structure. In this scenario, the algorithm finds a match on the first comparison and terminates immediately.
Worst-Case Situation	Occurs when the target element is located at the last index (n - 1) of the collection, or when the target element does not exist in the system at all. In both cases, the algorithm is forced to traverse through and compare every single element in the structure.
Time Complexity & Why	Best-Case Time Complexity: $O(1)$ — Requires only one comparison since the target is at the beginning. Worst-Case / Average Time Complexity: $O(n)$ — Why: The execution time grows linearly with the number of elements in the list. Since the search checks elements sequentially one by one from start to end, inspecting n elements requires up to n comparisons in the worst-case scenario.

SECTION 5 – TWO-DIMENSIONAL ARRAY / MATRIX (Requirement 5)

What the Rows Represent	Each row represents an individual primary entity record in the system (e.g., individual Student Records or Category Entries) mapped sequentially by row index.
What the Columns Represent	Each column represents specific numerical attributes or periodic dynamic metrics associated with each row entity (e.g., test scores across multiple grading periods or monthly stock quantities).
Matrix Operation 1 (name + explanation)	Row-wise Aggregation / Average Calculation: Iterates through all columns of a specific row using nested loops to compute total cumulative values or compute performance averages per entity record. Logic: <code>sum = 0; for(int j = 0; j < COLS; j++) sum +=</code>
Matrix Operation 2 (name + explanation)	Matrix Traversal & Search / Extremum Search: Scans all row and column elements to find specific values, or determine global maximum/minimum entries across all records. Logic: Uses two nested for loops (i and j) to compare each cell <code>matrix[i][j]</code> against target search criteria.

SECTION 6 – ALGORITHM DOCUMENTATION (Part VII)

Complete this table for each major operation in your system.

Operation	Input	Process	Output	Algorithm	Time Complexity
Student Insertion	Student details (ID, Name, Course, GPA)	Validates array capacity bounds (<code>currentCount < MAX</code>). Places new record at index <code>currentCount</code> and increments <code>currentCount++</code>	Success message ("Student added successfully") or error message if full.	Array Append / Dynamic Allocation	$O(1)$ (un-sorted append) or $O(n)$ if dynamic resize occurs
Student Search	Target Student ID	Iterates through array from index 0 to <code>currentCount - 1</code> , comparing target ID with stored student record IDs	Displays student details if found, or "Student Not Found" error message.	Linear Search	$O(n)$
Student Update	Target Student ID & newstudent fields (Name, Grades)	Performs Linear Search to locate record index. If located, directly updates fields using subscript assignment.	Confirmation message ("Record updated successfully").	Search-and-Update (Direct Assignment)	$O(n)$ (due to search phase)
Student Deletion	Target Student ID	Locates student index via Linear Search. Shifts subsequent elements leftward (i to $i - 1$) to fill gap, then decrements <code>currentCount--</code>	Confirmation message ("Student record deleted").	Array Element Shifting	$O(n)$
Array Traversal	None (or display filter choice)	Runs a for loop starting at index 0 up to <code>currentCount - 1</code> to read and format each record sequentially.	Formatted list/table of all active student records on screen.	Sequential Traversal	$O(n)$
Matrix Processing	2D Array of Student Grades/Scores	Uses nested for loops (rows i , columns j) to sum rows/columns and compute cumulative grade averages.	Displayed average grades or computed summary statistics per student.	Nested Loop Matrix Traversal	$O(n \times m)$ or $O(n^2)$

SECTION 7 – BIG-O ANALYSIS (Part VIII)

Explain, in your own words, why each stated complexity applies to your implementation.

Operation	Time Complexity	Explain Why
Direct array access	$O(1)$	Accessing an element directly by its index takes constant time because the computer calculates the exact memory location using a simple mathematical offset, regardless of the array size.
Array traversal	$O(n)$	Visiting or processing each element in a 1D array of size n requires executing the loop once per element, making the execution time grow linearly with n .
Linear search	$O(n)$	In the worst-case scenario, the target element is at the very end of the array or not present at all, requiring a check of all n elements sequentially.
Matrix traversal ($n \times n$)	$O(n^2)$	Iterating through a 2D matrix of size $n \times n$ requires two nested loops, executing the inner loop n times for each of the n rows, resulting in $n \times n = n^2$ total operations.

SECTION 8 – TEST CASES (Part IX, minimum of 10)

No.	Feature	Input	Expected Output	Actual Output	Status
1	Direct Array Access	arr[0] (Array:arr[0] (Array: [10, 20, 30])	10	10	passed
2	Direct Array Access	arr[2] (Array: [10, 20, 30])	30	30	passed
3	Out of Bounds Check	arr[5] (Array size: 3)	Error: Index out of range	Error: Index out of range	passed
4	Array Traversal	Print elements of [5, 15, 25]	5 15 25	5 15 25	passed
5	Linear Search (Existing Element)	Target: 20 in [10, 20, 30, 40]	Element found at index 11	Element found at index 11	passed
6	Linear Search (Missing Element)	Target: 99 in [10,20, 30, 40]	Element not found	Element not found	passed
7	Dynamic Memory Allocation	Allocate array of size 5	Memory allocated successfully	Memory allocated successfully	passed
8	Dynamic Memory Deallocation	Free allocated array memory	Memory freed without leakage	Memory freed without leakage	passed
9	2D Array / Matrix Access	matrix[0][1] in [[1,2], [3,	2	2	passed
10	Matrix Traversal	Traverse 2x2 matrix [[1,2],	1 2 3 4	1 2 3 4	passed

SECTION 9 – SUBMISSION CHECKLIST

- ☐ ADT identified (Section 1 complete)
- ☐ Static array implemented and documented (Section 2)
- ☐ Dynamic memory implemented and documented (Section 3)
- ☐ Searching implemented and documented (Section 4)
- ☐ 2D array / matrix implemented and documented (Section 5)
- ☐ Algorithm documentation completed (Section 6)
- ☐ Big-O analysis completed (Section 7)
- ☐ At least 10 test cases completed (Section 8)
- ☐ GitHub repository prepared with required structure
- ☐ README completed

- ☐ Screenshots included
- ☐ All group members can individually explain their assigned position
- ☐ Worksheet reviewed by all group members before submission

Group Reflection

What was the most difficult part of this assessment for your group, and how did you solve it?

The most challenging aspect of this assessment was ensuring proper dynamic memory management and accurately evaluating the time complexities associated with multi-dimensional array operations. Managing heap memory required meticulous handling to prevent memory leaks and dangling pointers, especially during structural reallocations. Additionally, distinguishing between theoretical time complexity and actual execution runtime across nested structures required deep analysis. Our group resolved these challenges by systematically conducting dry runs, utilizing debugging tools, rigorously testing edge cases, and holding peer-review sessions to ensure that both the algorithmic logic and documentation aligned seamlessly with the required specifications.

What would your group improve if you had more development time?

Given additional development time, our group would focus on enhancing the modularity and scalability of the implementation by integrating more advanced abstract data types and automated test scripts. We would also optimize the memory usage pattern for large-scale dataset processing and implement robust exception-handling mechanisms to manage unexpected user inputs gracefully. Furthermore, we would design a graphical user interface (GUI) or an interactive terminal dashboard to improve user experience, alongside implementing automated performance profiling to benchmark theoretical Big-O estimations against real-world system execution times.

“Build it. Test it. Explain it. Defend it.”